

Tarefa 6

Um jogo completinho

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis

PEDs: Caroline Nakazato, Giovanne Mariano, Gustavo Moraes e
Marcos Freire

PADs: Caio, Lia, Paulo e Vinicius

1. Descrição Geral

Ao longo dos últimos 5 laboratórios, você desenvolveu um jogo complexo, divertido e extensível por meio da utilização dos princípios de programação orientada a objetos. Além disso, a adição de testes de unidade, sistema de compilação via Gradle, e documentação via Javadoc tornou o projeto mais compreensível e fácil de manter. Ao longo deste processo, você também pôde exercitar sua criatividade por meio da construção de uma temática e do design de cartas, efeitos, inimigos e heróis.

O objetivo deste laboratório é fechar a sua visão para o jogo por meio da adição de elementos de progressão integrados ao mapa desenvolvido no laboratório passado. Estes elementos devem tornar cada partida única. Este laboratório será mais aberto do que os demais, com foco na integração de diferentes padrões de design para a construção de sistemas à sua escolha.

Assim como nos laboratórios anteriores, você tem liberdade para expandir e adaptar seu projeto, podendo utilizar quaisquer recursos da linguagem Java, incluindo bibliotecas externas gerenciadas via Gradle.

2. Objetivos

Ao final deste laboratório, o aluno deverá ser capaz de:

- Modelar sistemas mais complexos envolvendo **múltiplas entidades e interações**, como mapas com diferentes tipos de eventos e progressão entre batalhas;
- Projetar e implementar **mecanismos de progressão** que modificam o estado do jogador ao longo do jogo (ex: recompensas, loja, fogueira, etc.);
- Utilizar e integrar **padrões de projeto** na implementação de novos sistemas, compreendendo seu papel na organização e extensibilidade do código;
- Integrar diferentes componentes do sistema (batalha, mapa e eventos) de forma coesa e consistente;
- Documentar claramente o funcionamento dos sistemas implementados no **README**, incluindo decisões de modelagem e uso de padrões;
- Utilizar **diagramas UML** como ferramenta de apoio para planejar, visualizar e comunicar a estrutura do sistema.

3. Atividades

Esta seção descreve as atividades do laboratório. O objetivo é fechar o jogo com elementos de progressão entre múltiplas batalhas. Estes elementos serão integrados com o mapa desenvolvido no laboratório passado.

Assim como nos laboratórios anteriores, as instruções abaixo definem os requisitos mínimos esperados. Você é livre para adaptar e expandir sua implementação, desde que respeite os objetivos propostos.

As seguintes diretrizes gerais devem ser observadas:

- **Princípios de POO:** Continue utilizando herança, polimorfismo e encapsulamento para organizar seu código. Evite duplicação desnecessária e favoreça reutilização;
- **Organização do Projeto:** Mantenha o projeto estruturado de forma clara dentro do padrão do Gradle;

- **Liberdade Técnica:** Você pode utilizar qualquer recurso da linguagem Java, incluindo bibliotecas externas adicionadas via Gradle;
- **Autonomia:** Algumas decisões de modelagem são intencionalmente deixadas em aberto. Escolha soluções coerentes com o restante do seu projeto.

3.1. Extensão do Sistema de Mapa e Progressão



Figura 1: O mapa em Slay the Spire. Veja que ele é organizado como uma árvore em que cada nó representa um combate, loja ou evento genérico.

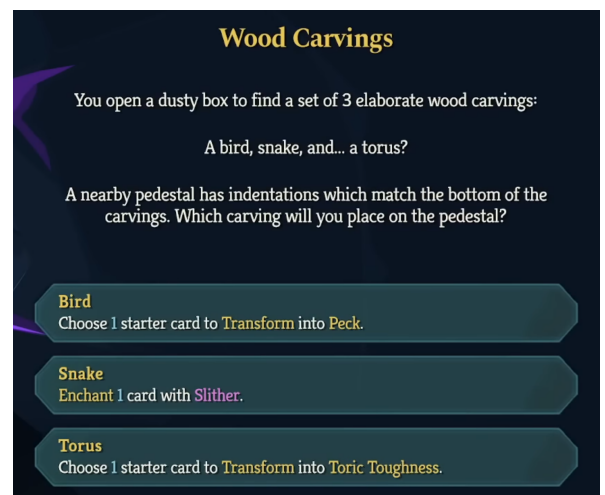
No laboratório anterior, foi implementado um sistema de mapa que conecta diversas batalhas. Queremos permitir que o jogador fique mais forte entre batalhas, como por meio da adição de cartas ao seu baralho ou de outras melhorias. Efetivamente, são salvos entre batalhas:

- O estado do jogador, incluindo seu total de vida mas não seus efeitos;
- As cartas no baralho do jogador;
- Outros elementos, a depender da implementação feita na Seção 3.2.

O sistema de mapa deve ser expandido para permitir a adição de eventos, como as lojas e fogueiras mostradas na Figura 1. Estes eventos, de forma genérica, devem ser capazes de fazer modificações no estado do jogador, da mesma forma como uma batalha pode resultar na perda de vida. Os sistemas implementados aqui servirão como base para a implementação desenvolvida na Seção 3.2.



(a) Uma recompensa após uma batalha vencida.



(b) Uma escolha.

Figura 2: Sistemas que afetam o estado do jogador em Slay the Spire

3.1.1. Classe abstrata Evento

Representam eventos genéricos no mapa, como batalhas e lojas.

■ Métodos mínimos

- **iniciar** — Recebe o estado do jogador e roda o evento, seja ele uma batalha, escolha, ou qualquer outro. O sistema de mapa deve ser capaz de verificar se o jogador perdeu o jogo a cada evento, seja por um retorno deste método ou por uma verificação do estado do jogador.

3.1.2. Classe Batalha

Classe implementada no laboratório passado. Deve passar a herdar de evento. Além disso, deve passar a **oferecer uma recompensa ao jogador** após ser vencida (Figura 2a). Em Slay the Spire esta recompensa vem na forma de:

- Uma carta adicionada ao seu baralho, escolhida dentre 3 opções aleatorizadas. O jogador pode pular esta escolha caso nenhuma das cartas seja do seu interesse;
- Ouro, para gastar na loja;
- Uma poção aleatória, que pode ou não ser ganha como recompensa;
- Uma relíquia aleatória, mas somente em combates especialmente difíceis.

Sinta-se livre para implementar esta recompensa como quiser, mas é **fundamental que o jogador seja oferecido alguma vantagem ao vencer cada batalha**, seja na forma de cartas novas ou atrelado a algum sistema implementado na Seção 3.2.

3.1.3. Classe Escolha

Herda de evento. Representa uma escolha que o jogador deve fazer no terminal que pode resultar em benefícios e malefícios. Um exemplo de escolha é mostrado na Figura 2b. Escolhas são uma oportunidade de dar personalidade para o jogo por meio da escrita de histórias divertidas, mas o único requisito do laboratório é que uma escolha possa ser feita que resulte em consequências **claras**. As consequências de uma escolha podem ser:

- Adição ou remoção de uma carta ao baralho;
- Perda ou recuperação de vida;
- Interação com algum dos sistemas implementados na Seção 3.2.

3.2. Elementos de Progressão

Deverão ser implementados sistemas de progressão para tornar o jogo mais interessante e dinâmico. Serão apresentadas algumas sugestões de sistemas, como uma loja que permite que o jogador gaste ouro obtido em batalhas ou uma fogueira na qual o jogador pode melhorar suas cartas ou recuperar sua vida, mas você também tem a liberdade de implementar sistemas criados por você. Neste laboratório, **você deve implementar dois sistemas**.

3.2.1. Requisitos de Implementação

São requisitos para cada sistema implementado:

- A descrição do sistema no README do projeto em uma seção a parte;
- A seleção de um **padrão de design** como base para fazer a implementação deste sistema:
 - O padrão de design pode ser tirado do refactoring.guru ou de alguma outra fonte online;
 - Tanto o nome do padrão de design quanto a fonte consultada devem estar no README junto com a descrição do sistema.
- O sistema deve estar relacionado à progressão entre batalhas, mudando o estado do jogador;
- Estes sistemas devem ter integração com os sistemas da Seção 3.1, seja como um novo tipo de recompensa em batalhas e escolhas, ou como um novo tipo de evento no mapa.



(a) A loja troca ouro por itens.



(b) Descanse ou melhore cartas na fogueira



(c) Relíquias dão melhorias passivas.



(d) Poções podem ser usadas durante batalhas.

Figura 3: Sistemas relacionados à progressão em Slay the Spire 2

3.2.2. Sugestões de Sistema

Você pode implementar qualquer sistema que siga os requisitos explicitados na Seção 3.2.1, incluindo invenções suas. Esta seção apresenta algumas sugestões de sistemas que fazem parte de Slay the Spire que podem ser implementados de modo a cumprir os requisitos deste laboratório.

Loja (Figura 3a) Um **evento no mapa** que permite que o jogador compre relíquias e cartas de uma seleção aleatória. Também permite que o jogador pague para remover cartas do seu baralho. A moeda de compra do jogo é o ouro, que pode ser obtido ao vencer batalhas ou como consequência de escolhas feitas.

Fogueira (Figura 3b) Um **evento no mapa** que permite que o jogador descanse, recuperando 30 % da sua vida máxima, ou melhore uma de suas cartas. Em Slay the Spire, toda carta tem uma versão normal e uma versão melhorada, que se difere por meio de modificações como aumento de dano causado, defesa recebida ou total de acúmulos de efeitos causados.

Relíquias (Figura 3c) Artefatos que podem ser comprados na loja ou recebidos como recompensas de batalhas ou consequências de escolhas. Ficam com o jogador permanentemente e conferem efeitos como:

- Começar cada combate com um efeito positivo;
- Recuperar vida toda vez que entrar em uma loja;
- Comprar uma carta ao levar dano durante uma batalha.

Algumas poucas relíquias devem ser implementadas para mostrar a flexibilidade do sistema, caso opte por implementá-las. Você pode usar a lista de relíquias na [wiki de Slay the Spire 2](#) como inspiração.

Poções (Figura 3d) compradas na loja ou recebidas como recompensas de batalhas ou consequências de escolhas. Podem ser utilizadas durante o combate, removendo-as permanentemente e oferecendo efeitos como:

- Recuperar vida do herói;

- Causar um efeito negativo no inimigo;
- Oferecer escudo ao herói no turno atual.

Algumas poucas poções devem ser implementadas para mostrar a flexibilidade do sistema, caso opte por implementá-las. Você pode usar a lista de poções na [wiki de Slay the Spire 2](#) como inspiração.

3.3. Pontos de Atenção

- Os sistemas implementados devem estar claramente descritos no README;
- Garanta que o funcionamento dos novos sistemas fica claro por meio da impressão no terminal.

3.4. Desafio Extra (+0,5 ponto extra)

Antes de implementar sistemas mais complexos, é comum utilizar diagramas para **planejar e visualizar a estrutura do código**. Uma das formas mais utilizadas para isso é o **diagrama de classes**, inspirado na *Unified Modeling Language* (UML).

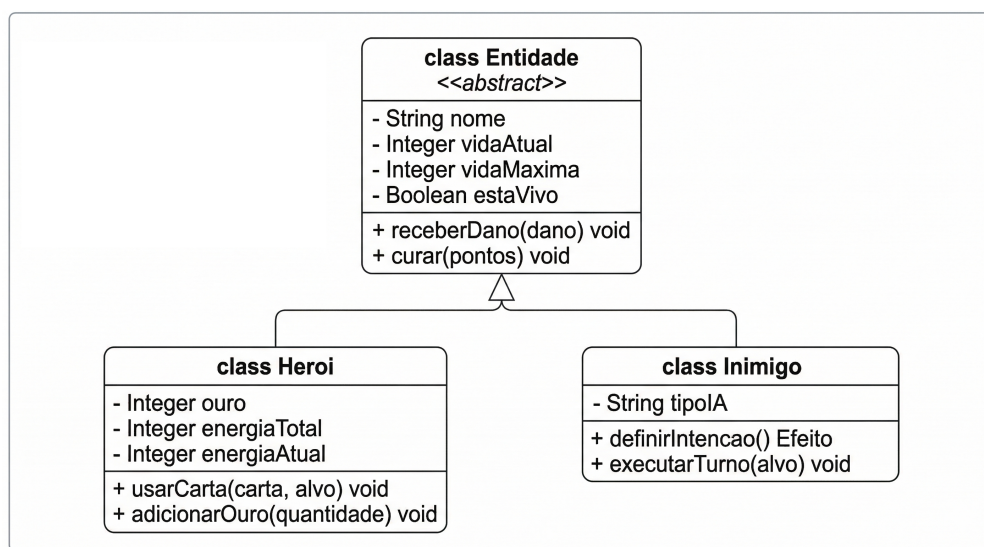


Figura 4: Exemplo de Diagrama de Classes UML.

Neste laboratório, não é necessário seguir rigorosamente toda a especificação formal da UML. O objetivo principal é utilizar diagramas como uma ferramenta prática para **organizar ideias**. Um diagrama simples já é suficiente, desde que represente:

- As classes envolvidas no sistema;
- Seus principais atributos e métodos;
- Os relacionamentos entre elas (herança, associação, uso, etc.).

Você pode criar seus diagramas utilizando ferramentas como o [draw.io](#), papel e caneta, ou qualquer outro meio de sua preferência.

3.4.1. O que deve ser modelado

Não é necessário diagramar todas as classes do projeto. Em vez disso, você deve focar em:

- Criar diagramas para os **padrões de projeto** utilizados nos sistemas implementados (Seção 3.2.1);
- Incluir esses diagramas no README, junto com a explicação do funcionamento de cada sistema.

O objetivo é demonstrar que você compreende como as classes se organizam para implementar o padrão escolhido.

3.4.2. Por que usar diagramas?

Criar diagramas antes da implementação ajuda a:

- Definir melhor **quais classes precisam existir**;
- Identificar **quais atributos e métodos** cada classe deve ter;
- Pensar com antecedência sobre **relações entre classes**, como herança e composição;
- Evitar retrabalho ao perceber problemas de modelagem ainda na fase de planejamento.

Em sistemas maiores, como o desenvolvido neste laboratório, essa etapa pode economizar tempo e resultar em um código mais organizado e fácil de manter.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. Sistemas de Progressão e Integração com o Mapa (4,0 pontos)

- Implementação de pelo menos dois sistemas de progressão (ex: loja, fogueira, relíquias, poções ou outros);
- Integração adequada desses sistemas com o mapa e com o fluxo do jogo;
- Os sistemas modificam corretamente o estado do jogador entre batalhas;
- Recompensas de batalhas são implementadas e influenciam a progressão;
- Funcionamento coerente e consistente dos sistemas implementados.

2. Boas Práticas de Desenvolvimento (4,0 pontos)

- Cada classe está em seu próprio arquivo `.java`;
- Código legível, bem organizado e com nomes claros;
- Uso adequado de orientação a objetos (encapsulamento, herança, polimorfismo, etc.);
- Documentação com **Javadoc** nas partes relevantes;
- Uso organizado de Git (commits descritivos e uso de branches apropriado);
- README bem estruturado, incluindo descrição dos sistemas implementados.

3. Funcionamento Geral e Experiência de Uso (2,0 pontos)

- O jogo executa corretamente do início ao fim, sem erros ou interrupções inesperadas;
- O fluxo entre mapa, eventos e batalhas é consistente;
- A interação via terminal é clara e compreensível;
- As informações exibidas permitem ao jogador entender o estado do jogo e tomar decisões.

4. Diagramas de Classes (+0,5 ponto extra)

- Inclusão de diagramas no README para os padrões de projeto utilizados;
- Diagramas representam corretamente as classes e suas relações;
- Os diagramas auxiliam na compreensão das decisões de modelagem.

5. Penalidade por Não Compilação

Caso o projeto:

- Não compile utilizando Gradle; ou
- Não execute corretamente no ambiente de correção;

será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

A nota final mínima é **0,0** (não haverá notas negativas).

5. Entrega

A submissão é realizada exclusivamente via repositório no **GitHub**. Para isso:

- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: **tarefaX-RA1-RA2**;
- Gere uma release no GitHub a partir dessa tag;
- Envie o link da release no Google Classroom.

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Caroline Nakazato – @CarolineNakazato
- Giovanne Mariano – @giolucasd
- Gustavo Morais – @moraguma
- Marcos Freire – @marcfreir

Lembrem-se que:

- O prazo de submissão é até o dia **29/04/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após 06/05/2026.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da raiz do repositório, para compilar e executar o projeto:

```
./gradlew build
./gradlew run
```

A classe principal do projeto deve conter o método:

```
public static void main(String[] args)
```

Essa classe será utilizada como ponto de entrada para a execução do programa. Certifique-se também de que ela está corretamente configurada no arquivo de build do Gradle para permitir a execução do comando **run**.

Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.